

Linux Process-Related Commands: fork, exec, ps, kill, and nice

These commands and system calls are fundamental to **process management** in Linux. They allow the operating system to create, execute, monitor, terminate, and prioritize processes.

1. **fork()** – Creating a New Process

Definition

fork() is a **system call** that creates a new process by duplicating the currently running process.

- The original process is called the **Parent Process**.
- The newly created process is called the **Child Process**.

The child process gets:

- A new Process ID (PID)
- A copy of the parent's memory
- Same program code
- Same environment variables
- Same open files

Initially, both processes execute the same code immediately after the fork() call.

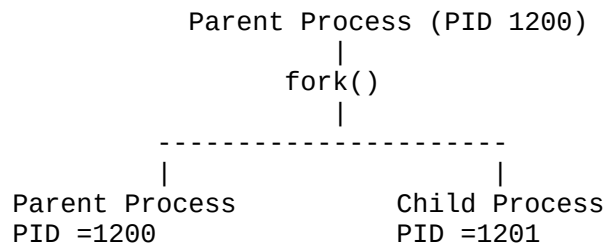
Syntax

```
pid_t fork(void);
```

Return Values

Return Value	Meaning
> 0	Parent process receives the Child PID
0	Child process
-1	Process creation failed

Process Creation Diagram



Common Uses

- Creating child processes
- Running multiple tasks simultaneously
- Servers handling multiple clients
- Shell creating commands

2. **exec()** Family – Executing Another Program

Definition

The **exec family** replaces the currently running program with another program.

Unlike `fork()`, **exec()** **does not create a new process**.

Instead,

```
Old Program
  ↓
exec()
  ↓
New Program
```

The Process ID remains the same.

Why use `fork()` with `exec()`?

Usually,

```
Parent
|
fork()
|
Child
|
exec()
|
Runs another program
```

Example:

Shell executes

```
ls
```

Internally

```
Shell
|
fork()
|
Child
|
exec(ls)
```

Common exec Functions

Function	Description
execl()	List of arguments
execv()	Argument array
execlp()	Searches PATH
execvp()	Searches PATH + array
execle()	Custom environment
execve()	Actual system call

Syntax

```
execl(path, arg0, arg1, ..., NULL);
```

Example

```
execl("/bin/ls", "ls", "-l", NULL);
```

Common Uses

- Shell commands
- Running another executable
- Process replacement
- Loading applications

Difference Between fork() and exec()

fork()	exec()
Creates new process	Replaces current process
Parent and child both exist	Old program disappears
New PID created	Same PID
Returns twice	Returns only on failure
Memory duplicated	Memory replaced

3. **ps** Command – Display Running Processes

Definition

ps stands for **Process Status**.

It displays information about currently running processes.

Basic Syntax

ps

Output

```
PID TTY TIME CMD
2154 pts/0 00:00:00 bash
2180 pts/0 00:00:00 ps
```

Important Options

1. **ps -e**

Displays every process.

```
ps -e
```

Example

```
PID TTY TIME CMD
1 ? 00:00 systemd
501 ? 00:00 sshd
902 ? 00:00 chrome
```

2. **ps -ef**

Shows complete details.

```
ps -ef
```

Columns

- UID
- PID
- PPID
- C
- STIME
- TTY
- TIME
- CMD

Example

```
UID PID PPID CMD
```

```
root 1 0 /sbin/init
ann 1250 1000 bash
ann 1456 1250 firefox
```

3. **ps -u**

Displays processes belonging to a user.

```
ps -u ann
```

4. **ps aux**

One of the most commonly used options.

```
ps aux
```

Displays

- CPU usage
- Memory usage
- Process owner
- Command

Example

```
USER PID %CPU %MEM COMMAND
```

```
root 1 0.2 0.5 systemd  
ann 1200 5.6 2.1 firefox
```

5. **ps -A**

Shows all processes.

```
ps -A
```

Equivalent to

```
ps -e
```

6. **ps -f**

Shows full-format listing.

```
ps -f
```

Includes

- UID
- PID
- PPID
- CMD

Important Fields

Field	Meaning
PID	Process ID
PPID	Parent Process ID
UID	User ID
CMD	Command
%CPU	CPU Usage
%MEM	Memory Usage
TTY	Terminal

4. **kill** Command – Terminating Processes

Definition

The `kill` command sends **signals** to processes.

Although its name is **kill**, it is used to send many different signals—not only to terminate a process.

Basic Syntax

```
kill PID
```

Example

```
kill 2345
```

Sends **SIGTERM (15)**.

Important Signals

Signal	Number	Purpose
SIGTERM	15	Graceful termination (default)
SIGKILL	9	Forcefully kill process
SIGSTOP	19	Pause process
SIGCONT	18	Resume process
SIGHUP	1	Reload configuration
SIGINT	2	Interrupt (Ctrl+C)

Examples

Gracefully terminate

`kill 2000`

Forcefully terminate

`kill -9 2000`

Stop process

`kill -STOP 2000`

Continue process

`kill -CONT 2000`

Interrupt process

`kill -2 2000`

Kill by Name

`pkill firefox`

Kills every Firefox process.

Kill Multiple Processes

```
kill 1001 1002 1003
```

Common Signals Summary

Command	Action
kill PID	SIGTERM
kill -9 PID	SIGKILL
kill -15 PID	SIGTERM
kill -19 PID	Stop
kill -18 PID	Continue

5. **nice** Command – Changing Process Priority

Definition

The **nice** command starts a process with a specified scheduling priority.

Linux scheduler uses the **nice value** to determine how much CPU time a process should receive.

Nice Value Range

-20 Highest Priority

0 Default

19 Lowest Priority

Lower nice value → Higher CPU priority.

Higher nice value → Lower CPU priority.

Syntax

```
nice -n value command
```

Example

Default priority

```
nice ls
```

Run with lower priority

```
nice -n 10 firefox
```

Run with highest priority (root only)

```
sudo nice -n -10 firefox
```

Viewing Nice Values

```
ps -l
```

Look at the **NI** column.

Example

```
PID NI CMD
```

```
2100 0 bash
```

```
2200 10 firefox
```

Changing Priority of Running Process

Using `renice`

```
renice 5 -p 2200
```

Changes process priority.

Nice Value Table

Nice Value	Priority
-20	Highest
-10	High
0	Default

10	Low
19	Lowest

Command/ System Call	Purpose	Common Syntax	Important Options/Varia nts
<code>fork()</code>	Creates a child process	<code>fork();</code>	Returns child PID to parent, 0 to child, -1 on failure
<code>exec()</code> family	Replaces the current process image with a new program	<code>execl("/bin/ls", "ls", "-l", NULL);</code>	<code>execl()</code> , <code>execv()</code> , <code>execlp()</code> , <code>execvp()</code> , <code>execle()</code> , <code>execve()</code>
<code>ps</code>	Displays process information	<code>ps</code>	-e, -A, -f, -ef, -u <user>, aux
<code>kill</code>	Sends signals to processes	<code>kill PID</code>	-9 (SIGKILL), -15 (SIGTERM), -STOP, -CONT, -2 (SIGINT)
<code>nice</code>	Starts a process with a specified priority	<code>nice -n value command</code>	-n <value> (nice range: -20 to 19); use <code>renice</code> to change the priority of a running process

These commands together form the foundation of Linux process management: **`fork()` creates processes, `exec()` loads programs, `ps` monitors them, `kill` controls or terminates them, and `nice` manages CPU scheduling priority.**